TripMate AI: Intelligent Group Activity & Travel Planning Platform

Powered by a Unified DeepSeek-V4 AI Agent Architecture

Technical Project Proposal for Academic Supervisor | Group Project Concept

Project Team (6 Members): Ta Quang Huy Ha Dang Huy Pham Dinh Anh Duong Nguyen Minh Duc Nguyen Tung Duong Dinh Tien Luan

1. Introduction, Scope & Product Overview

1.1. The Problem: Pain Points in Group Trip Planning

Coordinating leisure outings or vacations with friends is notoriously friction-heavy:

- **Tool Fragmentation:** Planning requires juggling 5+ disconnected applications (Google Maps, review blogs, Agoda, Booking.com, chat apps).
- **Preference Deadlocks:** Conflicting budgets, dietary restrictions, and activity styles stall decision-making.
- **Organizer Burnout & Buried Context:** Chat polls and venue links quickly get buried under hundreds of messages, leaving a single person overwhelmed with building and adjusting schedules.

1.2. Target Users & Practical Scope

- **Target Users:** Small groups (2–10 friends, university students, coworkers) organizing weekend city hangouts or multi-day vacations.
- **Practical Scope:** Domestic travel in Vietnam and visa-free ASEAN destinations, focusing strictly on intelligent scheduling, venue optimization, and real-time collaboration without visa complexities.

1.3. The Solution: Two-Stage Consensus Planning Flow

TripMate AI unifies group planning through a collaborative room powered by a **single, unified AI Agent (DeepSeek-V4)** that guides the group through two consensus checkpoints (Figure 1):

1. **Desires Capture:** Members share dates, budgets, and vibes in persistent room chat, saved directly into PostgreSQL.
2. **Macro Consensus:** The AI drafts high-level dates and daily themes; the group discusses and votes to lock the roadmap.
3. **Micro Curation & Stop Voting:** The AI queries live APIs (Google Places, Agoda, Klook) to populate venues clustered within a 2–3 km radius per half-day block. Members vote thumbs UP/DOWN, with 1-click alternative swaps.
4. **Final Confirmation:** The Host confirms the finalized plan, generating PDF and calendar (.ics) exports with synchronized Mapbox routes.

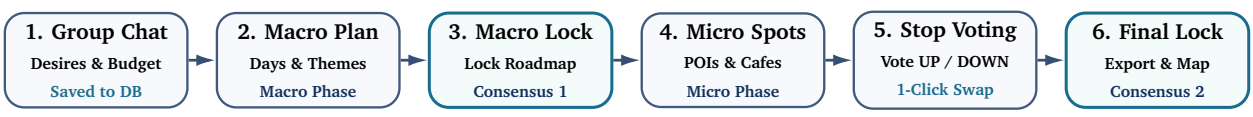

Figure 1: Two-stage consensus planning flow: gather desires → macro roadmap → lock macro → curate micro spots → granular vote → final plan lock

2. Platform Features & User Experience

2.1. Dual Planning Scopes

- **Day Hangouts:** Rapid curation for spontaneous outings (e.g., *"cafe → board game lounge → hotpot"*), prioritizing currently open venues with minimal transit times.

- **Multi-Day Vacations:** Complete travel itineraries (e.g., *"4 days in Da Nang & Hoi An"*) combining hotel recommendations, daily meal spots, sights, and optimized routes.

2.2. Shared Collaborative Room & Live Map Canvas

- **Invite via Link & Role Access:** Instant room entry with *Host* (plan locking, settings) and *Member* (chat, stop voting, swaps) permissions.
- **Interactive Timeline & Mapbox GL:** Draggable stop cards displaying photos, opening hours, estimated stay duration, and live Mapbox route navigation.

2.3. Two-Tier Consensus Engine

- **Stage 1 (Macro Approval):** Group locks the high-level roadmap before any expensive venue API calls are triggered.
- **Stage 2 (Micro Stop Voting):** Individual stops are voted UP/DOWN. Downvoted venues immediately present 2 pre-fetched backup options with 1-click swaps.
- **Living Itinerary:** Final plan confirmation persists normalized records to PostgreSQL but leaves the itinerary dynamic for real-time on-trip adjustments.

3. The AI Agent: Architecture, Workflow & Storage

3.1. Unified AI Agent: DeepSeek-V4 with Two-Phase State Execution

TripMate AI utilizes a **single, unified AI Agent powered by DeepSeek-V4** in LangGraph. To resolve latency bottlenecks and ensure clean testability, the agent operates in two decoupled state phases:

- **Phase 1 (Macro Planning State):** Proposes dates, pacing, and daily themes without invoking external travel APIs, conserving tokens until macro consensus is reached.
- **Phase 2 (Micro Curation State):** Activated only after macro approval. Executes external tool calls, applies 2–3 km geo-clustering to prevent zigzag routes, and prepares alternative stops.
- **Model Abstraction Layer:** Built using LangChain/LangGraph model abstractions. DeepSeek-V4 serves as the primary reasoning engine, with transparent fallback to OpenAI (GPT-4o), Anthropic Claude, or Google Gemini to prevent vendor lock-in.
- **Typed State Persistence:** Checkpoints of `AgentState` are saved in PostgreSQL via `AsyncPostgresSaver` through an authenticated Data Access Layer (DAL).

3.2. AI Agent Data Pipeline

Figure 2 illustrates the unified AI agent state and data pipeline, connecting room context, two-phase LangGraph states, typed checkpoints, model abstraction, and resilient tooling:

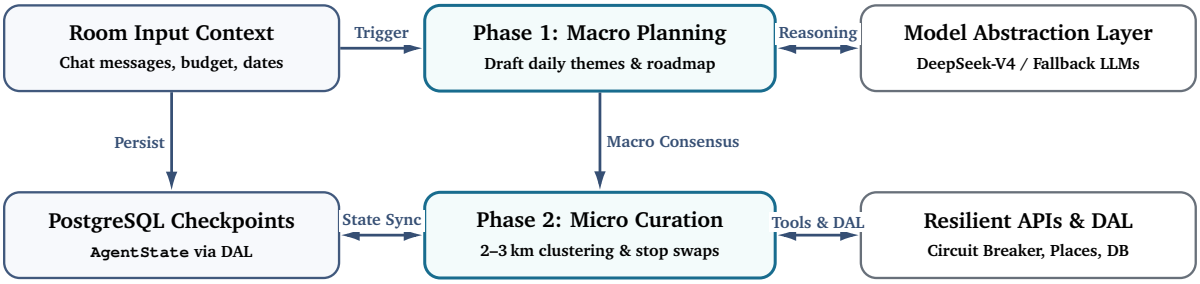

Figure 2: AI Agent state and data pipeline: two-phase LangGraph state execution with model abstraction and resilient tooling

3.3. Group Planning Workflow: Activity Diagram

Figure 3 illustrates the end-to-end user and AI interaction flow across two streamlined swimlanes:

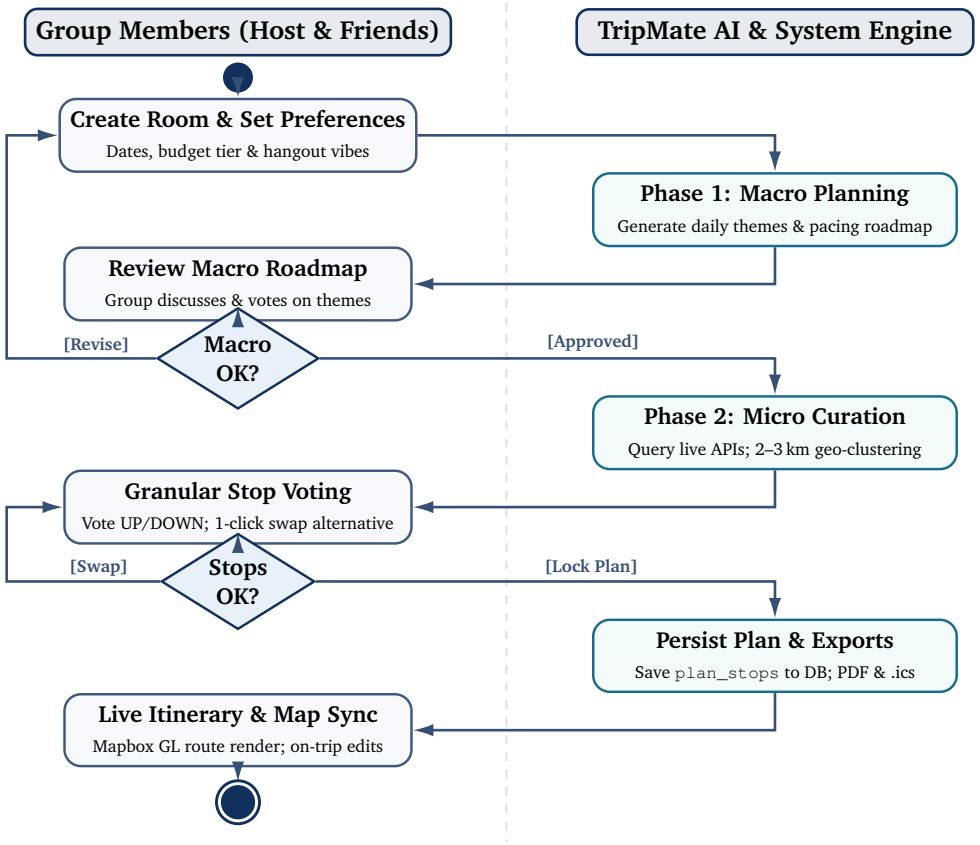

Figure 3: UML activity diagram: streamlined group planning workflow across user and AI system partitions

4. System Architecture & Operational Flow

TripMate AI uses an asynchronous, decoupled architecture connecting the React SPA, FastAPI gateway, PostgreSQL storage, the unified LangGraph AI agent, and external travel APIs. Figure 4 presents the complete system architecture and highlights the six primary communication interfaces (1)–(6) that coordinate the platform.

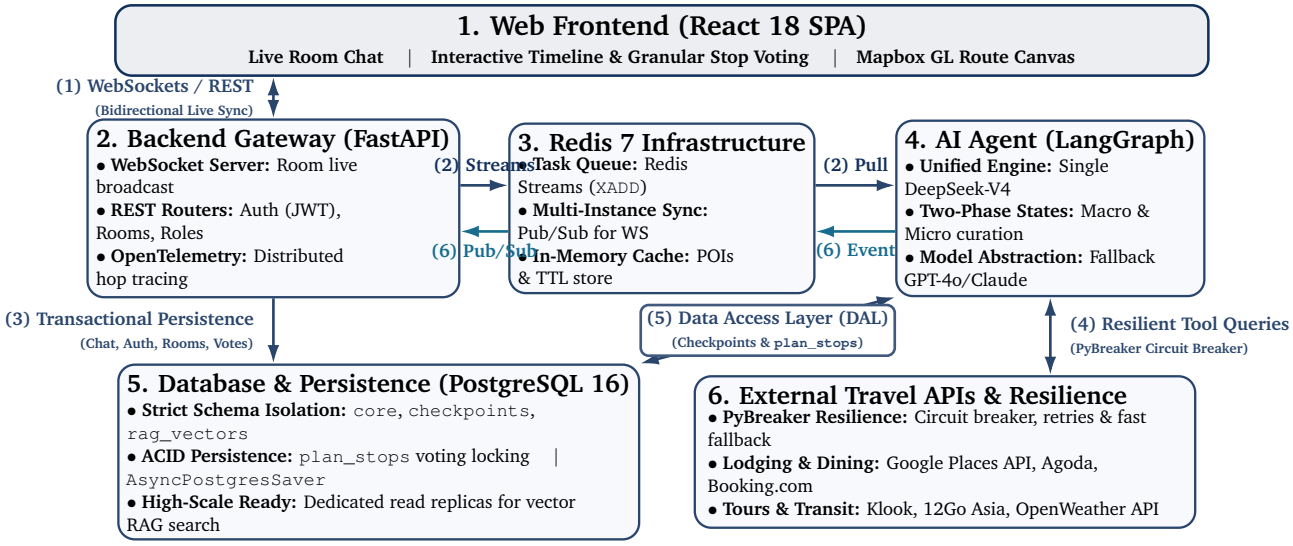

Figure 4: TripMate AI system architecture: component layers and operational interfaces (1)–(6)

4.1. End-to-End Operational Flow

Figure 5 visualizes the operational execution lifecycle of interfaces **(1)–(6)** from the System Architecture (Figure 4) as an end-to-end UML sequence diagram, showing the asynchronous decoupling of client interactions, background AI processing, resilient tool execution, and real-time room broadcasting. Table 1 summarizes each operational stage, execution mode, and dependent triggers.

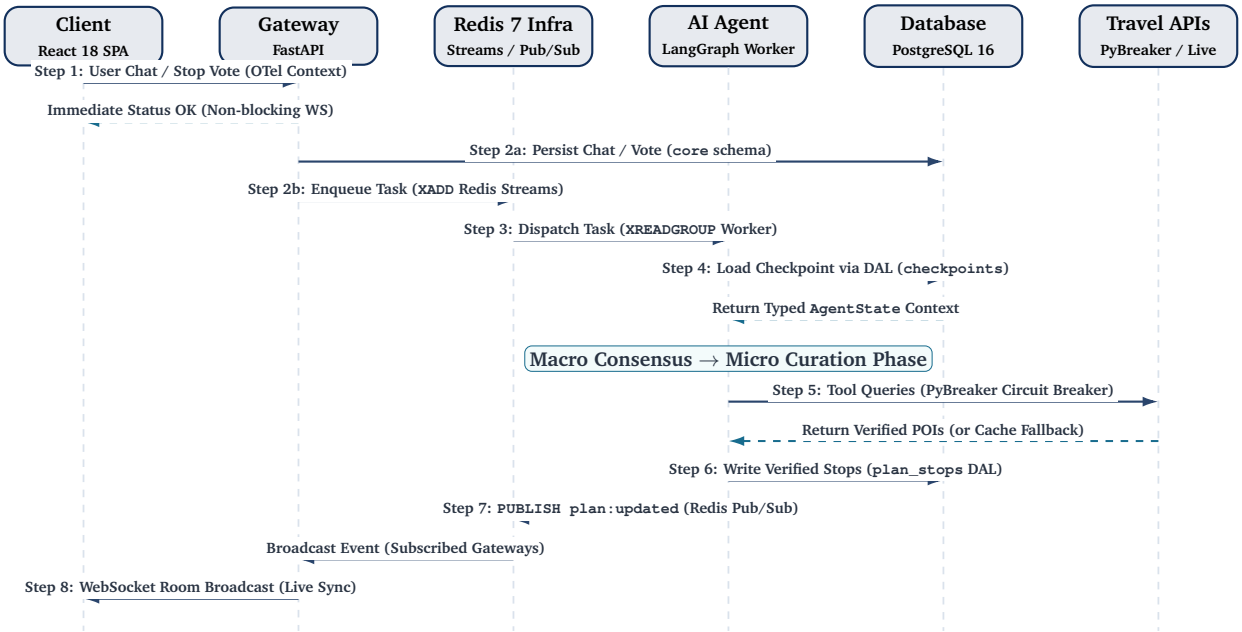

Figure 5: UML sequence diagram: end-to-end operational execution flow for the unified TripMate AI Agent

Step	Interface Path	Mode	Depends On	Operational Function
1	**(1)** Frontend → Gateway	Sync	User action	Transmits chat message or stop vote payload with OpenTelemetry context.
2a	**(3)** Gateway → Database	Sync	Step 1	Persists chat logs and vote counts into PostgreSQL (`core` schema).
2b	**(2)** Gateway → Redis (Streams)	Async	Step 1	Dispatches non-blocking task to Redis Streams (`XADD`); returns immediate HTTP/WS ack.
3	**(2)** Redis (Streams) → AI Agent	Async	Step 2b	LangGraph worker consumes task via `XREADGROUP` consumer group.
4	**(5)** AI Agent ← Database	Async	Step 3	Loads active session checkpoint and context via DAL from `checkpoints` schema.
4b	LangGraph State Transition	Internal	Step 4	Evaluates consensus and transitions between Macro Planning and Micro Curation phases.
5	**(4)** AI Agent ↔ Travel APIs	Parallel	Step 4b	Queries live APIs via PyBreaker circuit breaker; falls back to cache/vector gems on timeout.
6	**(5)** AI Agent → Database	Async	Step 5	Writes normalized, verified venue records to `plan_stops` via DAL.
7	**(6)** AI Agent → Redis → Gateway	Async	Step 6	Publishes `plan:updated` via Redis Pub/Sub to synchronize all FastAPI gateway instances.
8	**(1)** Gateway → Frontend	Push	Step 7	Broadcasts refreshed itinerary to all connected room members via WebSockets.

Table 1: Unified AI Agent operational execution flow mapped to system interfaces

Dynamic On-the-Go Adaptability: When conditions change unexpectedly (e.g., sudden rain), a member chats: *"it is raining, find an indoor cafe nearby"*. FastAPI immediately returns an acknowledgment (non-blocking WebSocket), records the message in the `core` schema (Step 2a), and enqueues an asynchronous task to Redis Streams (Step 2b). The LangGraph AI Agent worker consumes the task (Step 3), restores the active session checkpoint from PostgreSQL via DAL (Step 4), transitions directly to its Micro Curation state (Step 4b), queries nearby indoor venues in parallel through the resilient PyBreaker-guarded API and cache layer (Step 5), modifies **only the affected single row in `plan_stops`** in PostgreSQL (Step 6), publishes the update event to Redis Pub/Sub (Step 7), and FastAPI broadcasts the updated stop to all room members via WebSockets (Step 8).

4.2. Technology Stack

Tier	Core Technology	Architectural Role
Frontend	React 18 + TypeScript	Interactive timeline, collaborative voting cards, Mapbox GL canvas.
Backend Gateway	FastAPI (Python 3.11+)	REST endpoints, WebSocket room server, Redis Pub/Sub multi-instance sync.
AI Agent Layer	LangGraph Python	Unified DeepSeek-V4 agent with decoupled Macro and Micro state phases; typed `AgentState`.
	Model Abstraction	Primary: DeepSeek-V4; Fallback: OpenAI GPT-4o, Claude, Gemini.
Data & Messaging	PostgreSQL 16 + `pgvector`	Relational persistence, session checkpoints, and local RAG search (schema-isolated).
	Redis 7	Multi-instance WebSocket Pub/Sub, task message broker, and TTL cache.
Resilience & Ops	PyBreaker / OpenTelemetry	External API circuit breakers, timeout retries, and distributed hop tracing.

Table 2: Core technology stack for TripMate AI

4.3. Deployment, Security, Scalability & Observability

- **Decoupled Queue & Horizontal Scaling:** FastAPI gateways and LangGraph agent workers operate as decoupled containerized services communicating via Redis Streams. Gateway instances scale horizontally behind a load balancer, using **Redis Pub/Sub** to broadcast room updates across connected clients.

- **External API Resilience:** All queries to third-party providers (Google Places, Agoda, Klook) are guarded by **Circuit Breakers** (PyBreaker) with exponential backoff and Redis caching to prevent third-party latency from cascading.
- **Database Architecture & Isolation:** PostgreSQL manages transactional data, session checkpoints, and local vector embeddings within strictly isolated schemas (`core`, `checkpoints`, `rag_vectors`), preventing checkpoint writes and RAG queries from contending with transactional OLTP traffic.
- **End-to-End Observability:** Distributed tracing and structured logging are implemented via **OpenTelemetry** across the entire lifecycle (Frontend → Gateway → Queue → AI Agent → DB / APIs), with LangSmith tracing agent reasoning steps.
- **Security & Rate Limiting:** Passwords hashed with `bcrypt`, stateless JWT authentication, role-based room access control, and distributed Redis rate limiters protecting external API and LLM token budgets.

5. Expected Impact & Conclusion

TripMate AI transforms group trip planning:

- **Under 15 Minutes:** Replaces days of fragmented group chat debates with a structured, 15-minute consensus session.
- **Shared Ownership:** Granular stop voting and 1-click alternative swaps ensure every traveler has a voice without burning out a single organizer.
- **Accurate & Grounded:** Recommendations are verified against real-time data from Google Places, Agoda, and Klook, avoiding closed venues or unrealistic travel routes.
- **Dynamic & Resilient:** Normalized relational persistence enables instantaneous on-the-go plan revisions without recalculating unaffected stops.

TripMate AI turns group trip planning from an exhausting chore into an effortless, collaborative experience.